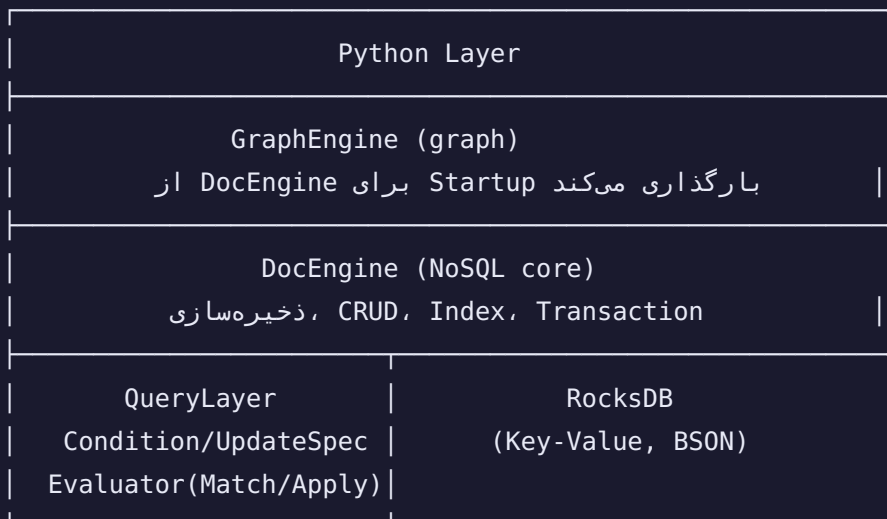


QueryLayer — NexoraDB

مستندات فنی سیستم Query

0.1.0	نسخه
Parser · GraphEngine · Python / Cython	تیم مخاطب
query/Condition.h · query/UpdateSpec.h · query/Evaluator.h · query/Evaluator.cpp	فایل‌های مرتبط

معماری صحیح سیستم



نکته مهم: QueryLayer وابستگی به RocksDB ندارد — به همین دلیل مستقل از DocEngine قابل تست است.

فهرست مطالب

۱. ساختار Condition
۲. ساختار UpdateSpec
۳. ساختار QueryOptions
۴. کلاس Evaluator
۵. راهنمای تیم Parser
۶. راهنمای تیم GraphEngine
۷. راهنمای تیم Cython
۸. جدول کامل عملگرها

۱. ساختار Condition

تعریف

```
// query/Condition.h
```

```

namespace nexora::query {

struct Condition {
    // — شرط ساده (leaf) —
    std::string field; // "age" / "address.city"
    Op op = Op::EQ;
    std::string value; // مقدار به صورت رشته
    ValueType value_type = ValueType::String; // نوع واقعی برای cast
    std::vector<std::string> values; // برای IN / NIN

    // — شرط ترکیبی (composite) —
    LogicOp logic = LogicOp::AND;
    std::vector<Condition> sub_conditions;

    // متدهای کمکی
    bool IsLeaf() const noexcept;
    bool IsEmpty() const noexcept; // همه اسناد match
    bool IsComposite() const noexcept;

    // Factory methods
    static Condition Leaf(field, op, value, value_type);
    static Condition In(field, values, negate=false);
    static Condition And(sub_conditions);
    static Condition Or(sub_conditions);
    static Condition Nor(sub_conditions);
};

```

نوع مقایسه	MongoDB معادل	C++ Enum
برابر	\$eq	Op::EQ
نابرابر	\$ne	Op::NEQ
بزرگتر	\$gt	Op::GT
بزرگتر یا مساوی	\$gte	Op::GTE
کوچکتر	\$lt	Op::LT
کوچکتر یا مساوی	\$lte	Op::LTE
عضو لیست	\$in	Op::IN
غیرعضو لیست	\$nin	Op::NIN
وجود/عدم وجود فیلد	\$exists	Op::EXISTS
الگوی regex	\$regex	Op::REGEX
شروع با پیشوند	—	Op::STARTS
حاوی substring	—	Op::CONTAINS

انواع ValueType

کاربرد	Enum
مقایسه رشته‌ای (پیش فرض)	<code>ValueType::String</code>
مقایسه عددی صحیح	<code>ValueType::Int64</code>
مقایسه عددی اعشاری	<code>ValueType::Float64</code>
مقایسه بولین	<code>ValueType::Bool</code>
فیلد null یا غایب	<code>ValueType::Null</code>

چرا `value_type` مهم است؟

```
age = "25" (string در doc)
cond.value = "9"
cond.op = Op::GT
```

(مقایسه رشته‌ای: `'9' > '2'` → `false`) بدون `value_type` → `"25" > "9"` → `false`
 (مقایسه عددی) با `value_type = Int64` → `25 > 9` → `true`

مثال‌های ساخت Condition

```
// ساده: age > 18
auto c1 = Condition::Leaf("age", Op::GT, "18", ValueType::Int64);

// ساده: username == "alice"
auto c2 = Condition::Leaf("username", Op::EQ, "alice");

// EXISTS: وجود داشته باشد avatar فیلد
auto c3 = Condition::Leaf("avatar", Op::EXISTS, "1");

// IN: status در این مقادیر باشد
auto c4 = Condition::In("status", {"active", "verified", "premium"});

// NIN: status در این مقادیر نباشد
auto c5 = Condition::In("status", {"banned", "deleted"}, /*negate=*/true);

// REGEX: username شروع شود با حرف a
auto c6 = Condition::Leaf("username", Op::REGEX, "^a");

// AND: age > 18 AND active == true
auto c7 = Condition::And({
    Condition::Leaf("age", Op::GT, "18", ValueType::Int64),
    Condition::Leaf("active", Op::EQ, "1", ValueType::Bool)
});

// OR: role == "admin" OR role == "moderator"
auto c8 = Condition::Or({
    Condition::Leaf("role", Op::EQ, "admin"),
    Condition::Leaf("role", Op::EQ, "moderator")
});

// ترکیبی پیچیده: (age >= 18 AND active) OR role == "admin"
auto c9 = Condition::Or({
    Condition::And({
        Condition::Leaf("age", Op::GTE, "18", ValueType::Int64),
        Condition::Leaf("active", Op::EQ, "1", ValueType::Bool)
    }),
    Condition::Leaf("role", Op::EQ, "admin")
});

// همه اسناد match خالی
Condition empty; // IsEmpty() = true
```

۲. ساختار UpdateSpec

تعریف

// query/UpdateSpec.h

```
namespace nexora::query {  
  
struct UpdateSpec {  
    std::vector<UpdateOperation> operations;  
    bool upsert = false; // پیاده‌سازی نشده MVP در  
  
    // Builder pattern – chain کردن:  
    UpdateSpec& Set(field, value, value_type);  
    UpdateSpec& Unset(field);  
    UpdateSpec& Inc(field, delta, value_type);  
    UpdateSpec& Push(field, element, value_type);  
    UpdateSpec& Pull(field, element, value_type);  
    UpdateSpec& TouchDate(field); // CurrentDate  
};
```

جدول عملگرهای UpdateOp

Enum	معادل MongoDB	رفتار
Set	\$set	مقدار فیلد را تنظیم/ایجاد می‌کند
Unset	\$unset	فیلد را از سند حذف می‌کند
Inc	\$inc	عدد delta را اضافه می‌کند (منفی = کاهش)
Mul	\$mul	مقدار را در factor ضرب می‌کند
Min	\$min	اگر مقدار جدید کمتر است جایگزین می‌کند
Max	\$max	اگر مقدار جدید بیشتر است جایگزین می‌کند
Rename	\$rename	نام فیلد را عوض می‌کند
CurrentDate	\$currentDate	Unix timestamp (ms) را می‌نویسد
Push	\$push	یک عنصر به آرایه اضافه می‌کند
PushAll	\$push \$each	چند عنصر اضافه می‌کند
Pull	\$pull	عناصر برابر با value را حذف می‌کند
PullAll	\$pullAll	عناصر موجود در لیست را حذف می‌کند
AddToSet	\$addToSet	فقط اگر عنصر نباشد اضافه می‌کند
Pop	\$pop	اول ("-1") یا آخر ("1") آرایه را حذف می‌کند

مثال‌های ساخت UpdateSpec


```
// سادہ builder
UpdateSpec spec;
spec.Set("bio", "Hello World")
    .Inc("likes", "1", UpdateValueType::Int64)
    .TouchDate("updated_at");

// مستقیم operations
UpdateSpec spec2;
spec2.operations.push_back(
    UpdateOperation::MakeSet("username", "new_alice")
);
spec2.operations.push_back(
    UpdateOperation::MakeInc("post_count", "-1", UpdateValueType::Int64)
);

// آرایہ
UpdateSpec spec3;
spec3.Push("tags", "sports")
    .Pull("tags", "gaming");

// PushAll
UpdateSpec spec4;
spec4.Add({UpdateOp::PushAll, "followers", "", {"u_001", "u_002", "u_003"},
    UpdateValueType::Array});

// Rename
spec4.Add(UpdateOperation::MakeUnset("old_field"));
// ل:
spec4.Add({UpdateOp::Rename, "old_name", "new_name", {}, UpdateValueType::String});
```

۳. ساختار QueryOptions

```
struct QueryOptions {
    uint32_t limit = 0;           // بدون محدودیت = 0
    uint32_t skip = 0;           // برای pagination
    SortSpec sort;               // مرتب‌سازی
    std::vector<std::string> projection; // فیلدهای خروجی
    bool projection_exclude = false; // false=include, true=exclude
};

struct SortSpec {
    std::vector<SortField> fields; // {field, SortOrder::Ascending/Descending}
};
```

مثال pagination

```
QueryOptions opts;
opts.limit = 20;
opts.skip = 40; // صفحه ۲ (zero-indexed) سوم
opts.sort.fields = {"created_at", SortOrder::Descending};
opts.projection = {"_id", "username", "avatar"}; // فقط این فیلدها
```

نکته: Sort در MVP توسط DocEngine انجام نمی‌شود. تیم Cython باید نتایج را در Python sort کند، یا DocEngine را در نسخه بعدی extend کند.

۴. کلاس Evaluator

ساخت

```
#include "query/Evaluator.h"

// MVP: پیش‌فرض JsonAdapter با
nexora::query::Evaluator evaluator;

// libbson (آینده، برای) سفارشی adapter با
auto evaluator = Evaluator(std::make_unique<LibBsonAdapter>());
```

Match(bson, condition) — تطابق شرط با سند

```
bool Match(const std::string& bson_doc, const Condition& condition) const;
```

رفتار	ورودی condition
همیشه true برمی‌گرداند	<code>IsEmpty()</code>
مقدار فیلد را با <code>op</code> مقایسه می‌کند	<code>IsLeaf()</code>
همه <code>sub_conditions</code> باید true باشند	AND <code>IsComposite()</code>
حداقل یک <code>sub_condition</code> کافی است	OR <code>IsComposite()</code>
هیچ‌کدام نباید true باشند	NOR <code>IsComposite()</code>
نقیض اولین <code>sub_condition</code>	NOT <code>IsComposite()</code>

مثال Match

```

Evaluator eval;

std::string doc = R"({"_id":"u1","age":25,"active":true,"role":"user"})";

auto c1 = Condition::Leaf("age", Op::GT, "18", ValueType::Int64);
bool ok1 = eval.Match(doc, c1); // true

auto c2 = Condition::And({
    Condition::Leaf("age", Op::GTE, "18", ValueType::Int64),
    Condition::Leaf("active", Op::EQ, "1", ValueType::Bool)
});
bool ok2 = eval.Match(doc, c2); // true

```

Update اعمال — Apply(bson, update_spec)

```
std::string Apply(const std::string& bson_doc, const UpdateSpec& spec) const;
```

سند اصلی تغییر نمی‌کند — یک کپی جدید برمی‌گرداند.

```

std::string old_doc = R"({"_id":"p1","likes":10,"tags":["tech"]})";

UpdateSpec spec;
spec.Inc("likes", "5", UpdateValueType::Int64)
    .Push("tags", "sports");

std::string new_doc = eval.Apply(old_doc, spec);
// new_doc = {"_id":"p1","likes":15,"tags":["tech","sports"]}

```

فیلتر فیلدها — ApplyProjection(bson, opts)

```
std::string ApplyProjection(const std::string& bson_doc, const QueryOptions& opts) const;
```

```

QueryOptions opts;
opts.projection = {"username", "email"};
// opts.projection_exclude = false → فقط این دو + _id

std::string projected = eval.ApplyProjection(doc, opts);

// exclude mode:
opts.projection = {"password", "token"};
opts.projection_exclude = true; // این دو را حذف کن

```

ExtractField(bson, field_path) — استخراج فیلد

```

FieldValue ExtractField(const std::string& bson_doc, const std::string& field_path) const

// FieldValue:
// .raw → مقدار به صورت رشته
// .type → ValueType
// .found → آیا فیلد وجود داشت؟

```

```

auto fv = eval.ExtractField(doc, "author_id");
if (fv.found) {
    graph.AddEdge(fv.raw, post_id, EdgeType::Authored);
}

// nested field:
auto city = eval.ExtractField(doc, "address.city");

```

۵. راهنمای تیم Parser

وظیفه تیم Parser

تیم Parser یک **DSL** یا **dict** پایتونی را به **Condition** یا **UpdateSpec** تبدیل می‌کند.

نمونه: تبدیل **Python dict** به **Condition**

```
# Python dict:
filter_dict = {
    "age": {"$gt": 18},
    "active": True
}

# Cython (در) باید تبدیل شود به:
cond = Condition.And([
    Condition.Leaf("age", Op.GT, "18", ValueType.Int64),
    Condition.Leaf("active", Op.EQ, "1", ValueType.Bool)
])
```

نگاشت MongoDB operators به Op enum

```
OP_MAP = {
    "$eq": Op.EQ,
    "$ne": Op.NEQ,
    "$gt": Op.GT,
    "$gte": Op.GTE,
    "$lt": Op.LT,
    "$lte": Op.LTE,
    "$in": Op.IN,
    "$nin": Op.NIN,
    "$exists": Op.EXISTS,
    "$regex": Op.REGEX,
}

UPDATE_OP_MAP = {
    "$set": UpdateOp.Set,
    "$unset": UpdateOp.Unset,
    "$inc": UpdateOp.Inc,
    "$mul": UpdateOp.Mul,
    "$min": UpdateOp.Min,
    "$max": UpdateOp.Max,
    "$push": UpdateOp.Push,
    "$pull": UpdateOp.Pull,
    "$addToSet": UpdateOp.AddToSet,
    "$pop": UpdateOp.Pop,
    "$rename": UpdateOp.Rename,
    "$currentDate": UpdateOp.CurrentDate,
}
```

نمونه: تشخیص **ValueType** از مقدار **Python**

```
def detect_value_type(val) -> ValueType:
    if isinstance(val, bool):    return ValueType.Bool
    if isinstance(val, int):     return ValueType.Int64
    if isinstance(val, float):   return ValueType.Float64
    if val is None:              return ValueType.Null
    return ValueType.String

def stringify(val) -> str:
    if isinstance(val, bool):    return "1" if val else "0"
    if val is None:              return "null"
    return str(val)
```

نمونه کامل parser در Cython

```

def parse_filter(filter_dict: dict) -> Condition:
    """تبدیل Python dict به Condition"""
    if not filter_dict:
        return Condition() # empty = match all

    subs = []
    for field, spec in filter_dict.items():
        if field == "$and":
            return Condition.And([parse_filter(s) for s in spec])
        if field == "$or":
            return Condition.Or([parse_filter(s) for s in spec])
        if field == "$nor":
            return Condition.Nor([parse_filter(s) for s in spec])

        if isinstance(spec, dict):
            # {"age": {"$gt": 18, "$lt": 65}} → AND ضمنی
            field_subs = []
            for op_str, op_val in spec.items():
                if op_str == "$in":
                    c = Condition.In(field, [str(v) for v in op_val])
                elif op_str == "$nin":
                    c = Condition.In(field, [str(v) for v in op_val], negate=True)
                else:
                    vt = detect_value_type(op_val)
                    c = Condition.Leaf(field, OP_MAP[op_str], stringify(op_val), vt)
                field_subs.append(c)
            if len(field_subs) == 1:
                subs.append(field_subs[0])
            else:
                subs.append(Condition.And(field_subs))
        else:
            # {"username": "alice"} → EQ
            vt = detect_value_type(spec)
            subs.append(Condition.Leaf(field, Op.EQ, stringify(spec), vt))

    if len(subs) == 1:
        return subs[0]
    return Condition.And(subs)

def parse_update(update_dict: dict) -> UpdateSpec:
    """تبدیل Python dict به UpdateSpec"""
    spec = UpdateSpec()
    for op_str, fields_dict in update_dict.items():
        op = UPDATE_OP_MAP.get(op_str)
        if op is None:
            continue
        if isinstance(fields_dict, dict):

```



```
for field, val in fields_dict.items():
    vt = detect_value_type(val)
    if op in (UpdateOp.PushAll, UpdateOp.PullAll, UpdateOp.AddToSet):
        vals = [str(v) for v in (val if isinstance(val, list) else [val])]
        spec.Add(UpdateOperation(op, field, "", vals, UpdateValueType.Array))
    else:
        spec.Add(UpdateOperation(op, field, stringify(val), [], vt))

return spec
```

۶. راهنمای تیم GraphEngine

جایگاه QueryLayer در GraphEngine

GraphEngine می‌تواند مستقیماً از **Evaluator** استفاده کند — بدون رفتن از طریق DocEngine:

```

#include "core/DocEngine.h"
#include "query/Evaluator.h"
#include "query/Condition.h"

class GraphEngine {
    nexora::core::DocEngine&    doc_engine_;
    nexora::query::Evaluator    evaluator_; // stateless, thread-safe

    std::unordered_map<std::string, UserNode> user_nodes_;

public:
    void BuildGraph() {
        // فیلتر فقط کاربران فعال
        auto active_cond = nexora::query::Condition::Leaf(
            "active", nexora::query::Op::EQ, "1", nexora::query::ValueType::Bool
        );

        doc_engine_.IterateCollection("users",
            [&](const std::string& id, const std::string& bson) -> bool {
                // فیلتر در سطح GraphEngine
                if (!evaluator_.Match(bson, active_cond)) return true;

                // استخراج فیلدهای مورد نیاز
                auto username = evaluator_.ExtractField(bson, "username");
                auto created   = evaluator_.ExtractField(bson, "created_at");

                UserNode node;
                node.id       = id;
                node.username  = username.found ? username.raw : "";
                node.created_at = created.found ? std::stoll(created.raw) : 0;
                user_nodes_[id] = std::move(node);
                return true;
            }
        );
    }

    void OnUserUpdated(const std::string& user_id, const std::string& new_bson) {
        // کن sync می‌کند، گراف را update را یک DocEngine وقتی
        auto username = evaluator_.ExtractField(new_bson, "username");
        if (user_nodes_.count(user_id) && username.found) {
            user_nodes_[user_id].username = username.raw;
        }
    }
};

```

```
void GraphEngine::DiscoverEdges() {
    // DocEngine ها از FK پیدا کردن همه
    auto collections = doc_engine_.ListCollections();
    for (const auto& col : collections) {
        auto fks = doc_engine_.GetForeignKeys(col);
        for (const auto& fk : fks) {
            // در گراف edge یک نوع FK هر
            // مثال: posts.author_id → users._id = edge "Authored"
            doc_engine_.IterateCollection(col,
                [&](const std::string& id, const std::string& bson) -> bool {
                    auto ref_val = evaluator_.ExtractField(bson, fk.local_field);
                    if (ref_val.found && !ref_val.raw.empty()) {
                        AddEdge(id, ref_val.raw, fk.fk_name);
                    }
                    return true;
                }
            );
        }
    }
}
```

۴. راهنمای تیم Cython

تعریف در **.pxd**

```
# bindings/python/nexoradb.pxd
```

```

# distutils: language = c++
from libcpp.string cimport string
from libcpp.vector cimport vector
from libcpp cimport bool

cdef extern from "query/Condition.h" namespace "nexora::query":
    cdef enum class Op(unsigned char):
        EQ, NEQ, GT, GTE, LT, LTE, IN, NIN, EXISTS, REGEX, STARTS, CONTAINS

    cdef enum class ValueType(unsigned char):
        String, Int64, Float64, Bool, Null

    cdef enum class LogicOp(unsigned char):
        AND, OR, NOR, NOT

    cdef cppclass Condition:
        string field
        Op op
        string value
        ValueType value_type
        vector[string] values
        LogicOp logic
        vector[Condition] sub_conditions

        bool IsLeaf()
        bool IsEmpty()
        bool IsComposite()

        @staticmethod
        Condition Leaf(const string&, Op, const string&, ValueType)

        @staticmethod
        Condition In(const string&, vector[string], bool)

        @staticmethod
        Condition And(vector[Condition])

        @staticmethod
        Condition Or(vector[Condition])

cdef extern from "query/UpdateSpec.h" namespace "nexora::query":
    cdef enum class UpdateOp(unsigned char):
        Set, Unset, Inc, Mul, Min, Max, Rename, CurrentDate
        Push, PushAll, Pull, PullAll, AddToSet, Pop

    cdef enum class UpdateValueType(unsigned char):
        String, Int64, Float64, Bool, Null, Array

```

```

cdef cppclass UpdateOperation:
    UpdateOp op
    string field
    string value
    vector[string] values
    UpdateValueType value_type

cdef cppclass UpdateSpec:
    vector[UpdateOperation] operations
    bool upsert
    UpdateSpec& Set(const string&, const string&, UpdateValueType)
    UpdateSpec& Inc(const string&, const string&, UpdateValueType)
    UpdateSpec& Unset(const string&)
    UpdateSpec& Push(const string&, const string&, UpdateValueType)
    UpdateSpec& Pull(const string&, const string&, UpdateValueType)
    UpdateSpec& TouchDate(const string&)

cdef extern from "query/Evaluator.h" namespace "nexora::query":
    cdef cppclass Evaluator:
        Evaluator()
        bool Match(const string&, const Condition&)
        string Apply(const string&, const UpdateSpec&)

```

استفاده در **.pyx**

bindings/python/nexoradb.pyx

```

def find_many(self, str collection, dict filter_dict,
               int limit=0, int skip=0) -> list:
    cdef Condition cond = parse_filter(filter_dict)
    cdef DBResult result = self._engine.FindMany(
        collection.encode(), cond, limit, skip
    )
    if result.success:
        import json
        return json.loads(result.data.decode())
    raise RuntimeError(result.error_msg.decode())

def update_by_id(self, str collection, str doc_id, dict update_dict) -> int:
    cdef UpdateSpec spec = parse_update(update_dict)
    cdef DBResult result = self._engine.UpdateById(
        collection.encode(), doc_id.encode(), spec
    )
    if result.success:
        return int(result.data.decode())
    raise RuntimeError(result.error_msg.decode())

```

٨. جدول کامل عملگرها

Op (Condition)

Enum Value	عدد	کاربرد	نمونه مقدار value
EQ	0	برابری	"alice"
NEQ	1	نا برابری	"banned"
GT	2	بزرگ تر	Int64 + "18"
GTE	3	بزرگ تر/مساوی	Int64 + "18"
LT	4	کوچک تر	Float64 + "100"
LTE	5	کوچک تر/مساوی	Float64 + "100"
IN	6	عضو لیست	در values قرار می گیرد
NIN	7	غیر عضو لیست	در values قرار می گیرد
EXISTS	8	وجود فیلد	"1" (وجود) / "0" (عدم وجود)
REGEX	9	regex	"^alice"
STARTS	10	prefix	"ali"
CONTAINS	11	substring	"lic"

UpdateOp (UpdateSpec)

value_type	values	value	عدد	Enum Value
نوع مقدار	—	مقدار جدید	0	Set
Null	—	—	1	Unset
Int64/Float64	—	delta (منفی = کاهش)	2	Inc
Int64/Float64	—	factor	3	Mul
Int64/Float64	—	حداقل مجاز	4	Min
Int64/Float64	—	حداکثر مجاز	5	Max
String	—	نام جدید	6	Rename
Int64	—	—	7	CurrentDate
نوع عنصر	—	عنصر	8	Push
Array	لیست عناصر	—	9	PushAll
نوع عنصر	—	عنصر برای حذف	10	Pull
Array	لیست برای حذف	—	11	PullAll
نوع عنصر	—	عنصر (اگر نباشد اضافه می شود)	12	AddToSet
String	—	اول = "-1" / آخر = "1"	13	Pop

ضمیمه: تغییرات لازم در DocEngine.cpp

DocEngine.cpp یک placeholder برای Condition/UpdateSpec دارد که باید با include واقعی جایگزین شود:

حذف از DocEngine.cpp (بخش placeholder)

```
// حذف کنید این بخش را از DocEngine.cpp:
namespace nexora {
namespace query {
enum class Op : uint8_t { ... };
struct Condition { ... };
struct UpdateSpec { ... };
} // namespace query
} // namespace nexora
```

اضافه کردن به ابتدای DocEngine.cpp

```
#include "query/Condition.h"
#include "query/UpdateSpec.h"
#include "query/Evaluator.h"

// MatchesCondition و ApplyUpdate:
bool DocEngine::MatchesCondition(const std::string& bson,
                                const nexora::query::Condition& condition) {
    // thread_local مکرر از ساخت جلوگیری
    thread_local nexora::query::Evaluator eval;
    return eval.Match(bson, condition);
}

std::string DocEngine::ApplyUpdate(const std::string& bson,
                                   const nexora::query::UpdateSpec& spec) {
    thread_local nexora::query::Evaluator eval;
    return eval.Apply(bson, spec);
}
```